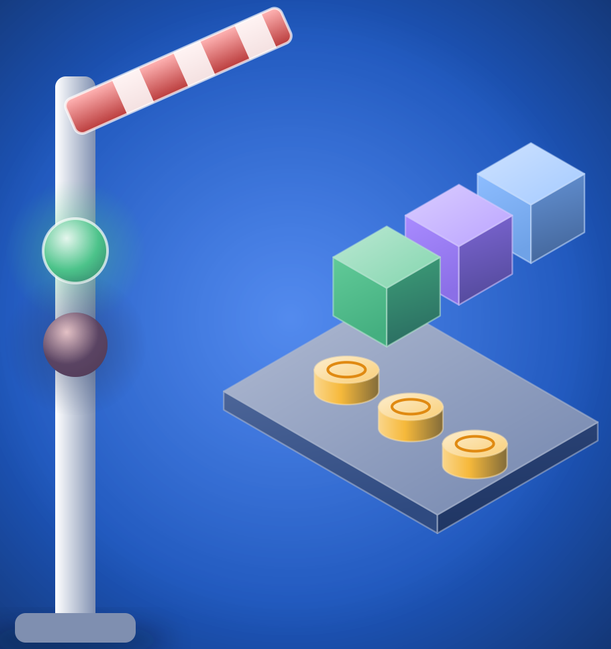
Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · WEEK 9

Synchronization I

Semaphores, binary and counting, how they are implemented, and the monitors and liveness problems that follow from them.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Learning objectives

1. Define a semaphore and its two operations.
2. Tell a binary semaphore from a counting semaphore, and use each.
3. Explain how a semaphore is implemented without busy waiting.
4. Describe monitors and condition variables.
5. Recognise liveness problems: deadlock, starvation and priority inversion.

THIS WEEK

Topic 7

Before the midterm we protected a critical section with a mutex lock. A lock answers one question: is it free? Many problems need more than that.

From Week 6

THE PROBLEM

Race condition

The result depends on the order in which concurrent processes access shared data.

THE REQUIREMENTS

Critical section

A solution needs mutual exclusion, progress and bounded waiting.

THE FIRST TOOL

Mutex lock

acquire() before the critical section, release() after. Built on atomic hardware instructions.

DISCUSSION

A car park has 50 spaces and one gate

A lock can say open or closed. How does the gate know when to let the next car in, and how many may enter?

PART 1

Semaphores

An integer that counts, and two operations that are atomic.

What a semaphore is

```
wait(S) {
    while (S <= 0)
        ; /* wait */
    S--;
}

signal(S) {
    S++;
}
```

A semaphore S is an integer variable.

After it is initialised, it is accessed only through two atomic operations: wait() and signal(). They were originally called P() and V().

wait() takes one unit, and waits if none is left. signal() gives one unit back.

This first definition waits by looping. We remove the loop later.

Counting and binary semaphores

0, 1, 2, … N

Counting semaphore

Its value ranges over any number. It controls access to a resource with a finite number of instances, and is initialised to that number. At 0, all instances are in use.

0 OR 1

Binary semaphore

Its value is only 0 or 1. It behaves like a mutex lock.

The car park: a counting semaphore initialised to 50. Each car does wait() at the gate on the way in and signal() on the way out.

Use 1: mutual exclusion

```
semaphore mutex = 1;

wait(mutex);
/* critical section */
signal(mutex);
```

A binary semaphore initialised to 1.

The first process to call wait() takes the value to 0 and enters. Any other process waits until signal() brings it back to 1.

Use 2: ordering two statements

```
semaphore synch = 0;

/* process P1 */
S1;
signal(synch);

/* process P2 */
wait(synch);
S2;
```

We want statement S2 to run only after S1 has finished.

The semaphore starts at 0. P2 cannot pass wait() until P1 has executed S1 and called signal().

A semaphore can therefore do something a mutex cannot: one process waits, and a different process releases it.

Use 3: a limited number of instances




```
semaphore printers = 5;

wait(printers); /* take one */
/* use a printer */
signal(printers); /* give it back */
```

Five identical printers. The semaphore's value is the number still free: here 2.

A sixth process that calls wait() while all five are in use waits until one is released.

PART 2

Implementing semaphores

wait() and signal() are themselves critical sections.

The implementation problem



Atomic

No two processes may execute wait() and signal() on the same semaphore at the same time. The implementation is its own critical-section problem.



Busy waiting

The simple definition loops while it waits. That wastes the CPU if a critical section is long, or if many processes are waiting.



The better way

Put the waiting process to sleep, and wake it when the semaphore becomes available.

A semaphore with a waiting queue

```
typedef struct {
    int value;
    struct process *list;
} semaphore;

wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        /* add this process to S->list */
        sleep();
    }
}
```

```
signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        /* remove a process P from S->list */
        wakeup(P);
    }
}
```

sleep() suspends the calling process and puts it on the semaphore's queue. wakeup(P) moves process P back to the ready queue.

Reading the value

3

Positive

Three instances are free. A wait() succeeds at once.

0

Zero

Nothing is free, and nobody is waiting.

−2

Negative

Nothing is free, and two processes are asleep on the queue.

In this implementation the value may become negative. Its magnitude is then the number of processes waiting.

What can go wrong with semaphores

The mistake	The code	The result
The two operations in the wrong order	signal(mutex) … wait(mutex)	Several processes in the critical section at once
wait() written twice	wait(mutex) … wait(mutex)	The process blocks for ever
wait() left out	… signal(mutex)	Mutual exclusion is violated
signal() left out	wait(mutex) …	Every other process blocks for ever

One mistake in one process is enough, and the error shows only with certain timings. This is the reason for a higher-level construct.

PART 3

Monitors

Let the programming language enforce mutual exclusion.

What a monitor is

```
monitor Account {
    int balance;

    procedure deposit(int n) {
        balance = balance + n;
    }

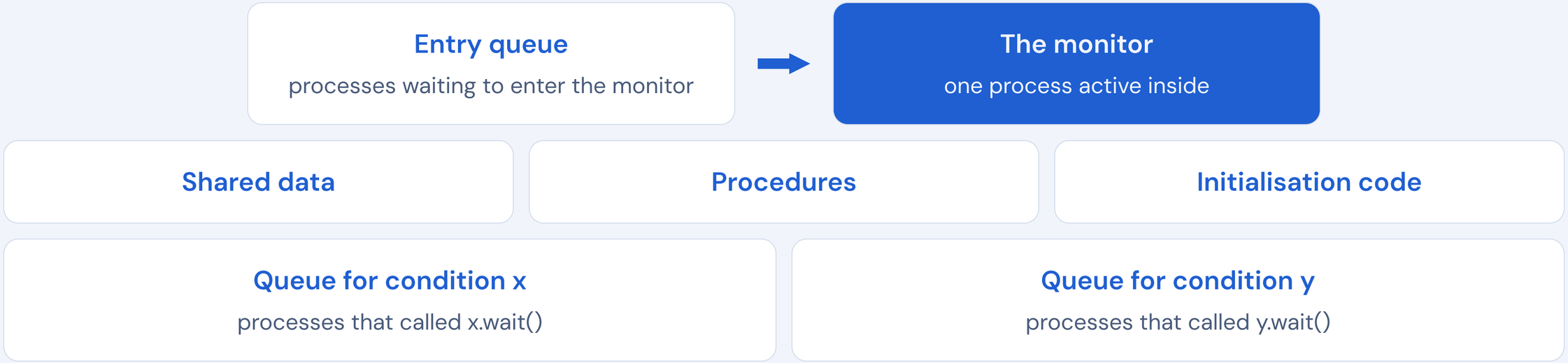
    procedure withdraw(int n) {
        if (balance >= n)
            balance = balance - n;
    }
}
```

An abstract data type: shared data, with the procedures that operate on it.

The data can be reached only through the monitor's procedures.

Only one process at a time can be active inside the monitor. The compiler adds the locking, so the programmer cannot forget it.

A view of a monitor



Mutual exclusion alone is not enough. A process inside the monitor sometimes has to wait for something to become true. That is what condition variables are for.

Condition variables

WAIT

x.wait()

The process that calls it is suspended, and leaves the monitor free for others, until another process calls x.signal().

SIGNAL

x.signal()

Resumes exactly one process that is waiting on x. If no process is waiting, it has no effect at all.

This is the difference from a semaphore. signal() on a semaphore always changes its value. signal() on a condition with nobody waiting is lost.

After a signal, who runs?

01

Signal and wait

P, which signalled, waits. Q, which was waiting, runs until it leaves the monitor or waits again.

02

Signal and continue

P continues. Q waits until P leaves the monitor or waits for another condition.

03

A compromise

P leaves the monitor immediately after signalling, so Q resumes at once. Several languages do this.

P signals while Q is waiting on the condition. Both cannot be active in the monitor, so one of them must wait.

Ordering with a condition variable

```
/* in a monitor */
condition x;
bool done = false;

/* process P1 */
S1;
done = true;
x.signal();

/* process P2 */
if (!done)
    x.wait();
S2;
```

The same problem as before: S2 only after S1.

Because a signal with no waiter is lost, we also need the variable done.

If P1 runs first, done is true and P2 does not wait. If P2 runs first, it waits and P1's signal wakes it.

Building a monitor from semaphores

The monitor's lock

A semaphore mutex, initialised to 1. Each procedure does wait(mutex) on entry and releases it on exit.

Signalling processes

A semaphore next, initialised to 0, where a process that has signalled suspends itself, and a counter of how many are suspended there.

Each condition

A semaphore initialised to 0 and a counter of waiters. x.wait() and x.signal() are written with these.

Monitors add no power to semaphores. They add safety: the protocol is written once, by the compiler, and not in every program.

Which waiting process to resume

THE DEFAULT

First-come, first-served

The process that has waited longest on the condition is resumed. Often good enough.

BY PRIORITY

Conditional wait

x.wait(c), where c is a priority number. When x.signal() is called, the waiting process with the smallest number is resumed.

Example: one resource shared by competing processes. Each tells the monitor how long it plans to use the resource, and the shortest request is served first.

A monitor that allocates one resource

```
monitor ResourceAllocator {
    bool busy = false;
    condition x;

    procedure acquire(int time) {
        if (busy)
            x.wait(time);
        busy = true;
    }

    procedure release() {
        busy = false;
        x.signal();
    }
}
```

A process calls acquire(t), uses the resource, then calls release().

The monitor guarantees its own procedures are mutually exclusive. It cannot force processes to call them in the right order.

A process might use the resource without acquiring it, never release it, or release it twice. The same kinds of mistake as with semaphores, one level higher.

PART 4

Liveness

A system must not only be safe. Its processes must also make progress.

Liveness

A process may have to wait *indefinitely* to acquire a lock or a semaphore.

What it means

Liveness is the set of properties a system must satisfy so that its processes make progress.

A liveness failure

Waiting for ever violates the progress and bounded-waiting requirements of Week 6.

Three forms

Deadlock, starvation and priority inversion.

Deadlock

Step	P0	P1
1	wait(S)	
2		wait(Q)
3	wait(Q): blocked	
4		wait(S): blocked

Deadlock is the subject of Week 11.

S and Q are two semaphores, both initialised to 1.

P0 holds S and waits for Q. P1 holds Q and waits for S.

Each waits for an event that only the other can cause: a signal(). Neither will ever execute it.

Starvation and priority inversion



Starvation

Indefinite blocking. A process may never be removed from the semaphore's queue in which it is suspended, for example if the queue is served last in, first out.



Priority inversion

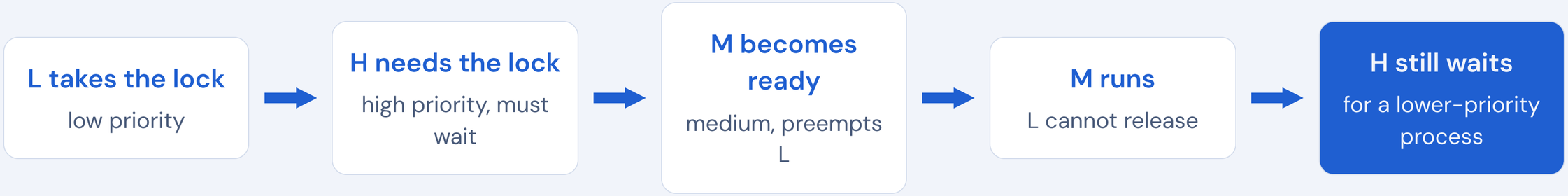
A low-priority process holds a lock that a high-priority process needs. A medium-priority process preempts the low one, so the high-priority process waits for both.



Priority inheritance

The solution: the process holding the lock temporarily inherits the priority of the highest-priority process waiting for it.

Priority inversion, step by step



With priority inheritance, L runs with H's priority while it holds the lock. M cannot preempt it, L finishes and releases the lock, and H runs next.

PART 5

The classical problems

Three small problems used to test every new synchronization scheme.

Three classical problems



Bounded buffer

A producer and a consumer share a buffer of fixed size. Neither may overrun the other.



Readers and writers

Many may read shared data at once, but a writer needs it alone.



Dining philosophers

Five processes, five resources, and each process needs two of them.

Week 10 solves each of them with the tools of today: semaphores and monitors.

Key takeaways

01

A semaphore counts

wait() takes a unit and may block. signal() gives one back and may wake a process.

02

Binary and counting

A binary semaphore is a lock. A counting semaphore guards a number of identical instances.

03

Monitors

Mutual exclusion written by the compiler, with condition variables for waiting inside.

04

Liveness

Deadlock, starvation and priority inversion all leave a process waiting without end.

Next session

COMING UP

Synchronization II

The producer–consumer, readers–writers and dining-philosophers problems, and synchronization in POSIX and Java.

SOURCE

Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapter 6, sections 6.6 to 6.8.